

Cross-Regression Primitive Extraction & Infrastructure Strain Physics Framework

Overview

Building a **meta-analytical system** that processes multiple pressure-based reports to extract **common primitives** and model **infrastructure strain dynamics** using physics-inspired force equations.

System Architecture

```
```python
import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import List, Dict, Tuple
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN
from scipy.stats import linregress
import networkx as nx
from collections import defaultdict
import re

@dataclass
class InfrastructureStrain:
 """Physics-based infrastructure strain model"""

 # Core strain dimensions
 legacy_framework_lag: float # Regulatory/policy obsolescence vs. technology deployment
 hidden_layer_governance: float # Unregulated market activity intensity
 evidence_gaps: float # Capital deployment vs. clinical/scientific backing
 fragmentation: float # System bifurcation / interoperability failure
 two_tier_outcomes: float # Access inequality / market segmentation
 narrative_collapse: float # Fear/threat dominance vs. response adequacy

 # Derived metrics
 total_strain: float
 criticality_score: float
 failure_probability: float

@dataclass
class PrimitiveCluster:
 """Clustered primitive patterns across reports"""
 cluster_id: int
 primitive_type: str # 'fear', 'threat', 'response'
 core_pattern: str
 frequency: int
 reports: List[str]
 tf_idf_score: float
 semantic_variants: List[str]

class CrossReportPrimitiveExtractor:
 """
 Extract common primitives across multiple pressure-based reports
 using TF-IDF, clustering, and pattern matching
 """
```

```

def __init__(self):
 self.vectorizer = TfidfVectorizer(
 max_features=500,
 ngram_range=(2, 5), # Capture multi-word primitives
 stop_words='english'
)
 self.primitive_clusters = {
 'fear': [],
 'threat': [],
 'response': []
 }

def extract_primitives_from_reports(
 self,
 reports: List[Dict[str, any]]
) -> Dict[str, List[PrimitiveCluster]]:
 """
 Process multiple reports and extract common primitives

 Args:
 reports: List of report dictionaries with structure:
 {
 'title': str,
 'url': str,
 'primitives': {
 'fear': List[str],
 'threat': List[str],
 'response': List[str]
 },
 'pressure_scores': Dict[str, float],
 'sector_analyses': List[...]
 }
 """

 # Process each primitive type separately
 for prim_type in ['fear', 'threat', 'response']:
 # Collect all primitives of this type
 all_primitives = []
 primitive_sources = []

 for report in reports:
 for primitive in report['primitives'][prim_type]:
 all_primitives.append(primitive)
 primitive_sources.append(report['title'])

 # TF-IDF vectorization
 if len(all_primitives) > 0:
 tfidf_matrix = self.vectorizer.fit_transform(all_primitives)

 # Cluster similar primitives
 clustering = DBSCAN(eps=0.3, min_samples=2, metric='cosine')
 clusters = clustering.fit_predict(tfidf_matrix)

 # Extract cluster patterns
 for cluster_id in set(clusters):
 if cluster_id == -1: # Skip noise

```

continue

```
cluster_indices = np.where(clusters == cluster_id)[0]
cluster_primitives = [all_primitives[i] for i in cluster_indices]
cluster_sources = [primitive_sources[i] for i in cluster_indices]

Find most representative primitive (highest TF-IDF)
cluster_tfidf_scores = tfidf_matrix[cluster_indices].sum(axis=1).A1
core_idx = cluster_indices[np.argmax(cluster_tfidf_scores)]

primitive_cluster = PrimitiveCluster(
 cluster_id=cluster_id,
 primitive_type=prim_type,
 core_pattern=all_primitives[core_idx],
 frequency=len(cluster_primitives),
 reports=list(set(cluster_sources)),
 tf_idf_score=float(np.max(cluster_tfidf_scores)),
 semantic_variants=cluster_primitives[:5] # Top 5 variants
)

self.primitive_clusters[prim_type].append(primitive_cluster)
```

return self.primitive\_clusters

class InfrastructureStrainPhysics:

"""

Model infrastructure strain using physics-inspired force equations

Conceptual Framework:

- Infrastructure = Material under stress
- Primitives = Forces acting on the material
- Strain = Deformation from equilibrium
- Failure = System breakdown/regime change

"""

```
def __init__(self):
 self.strain_history = []
```

```
def calculate_legacy_framework_lag(
```

```
 self,
 reports: List[Dict]
```

```
) -> float:
```

```
"""
```

$$F_{lag} = k_1 \times (\text{Technology\_Deployment\_Rate} - \text{Regulatory\_Adaptation\_Rate})^2$$

Measures: How fast is capital/technology deploying vs. regulatory framework updates?

Indicators:

- "Regulatory uncertainty" frequency
- "FDA framework disease-centric" patterns
- Capital deployment YoY growth vs. policy changes
- "Gray market" / "unregulated" mentions

"""

```
lag_indicators = {
 'regulatory_uncertainty': 0,
 'framework_obsolete': 0,
```

```

 'gray_market': 0,
 'policy_friction': 0
 }

 total_primitives = 0

 for report in reports:
 all_text = ''.join(
 report['primitives']['fear'] +
 report['primitives']['threat'] +
 report['primitives']['response']
).lower()

 total_primitives += len(report['primitives']['fear']) + \
 len(report['primitives']['threat']) + \
 len(report['primitives']['response'])

 # Pattern matching
 lag_indicators['regulatory_uncertainty'] += all_text.count('regulatory uncertainty')
 lag_indicators['regulatory_uncertainty'] += all_text.count('policy uncertainty')
 lag_indicators['framework_obsolete'] += all_text.count('framework') + all_text.count('outdated')
 lag_indicators['framework_obsolete'] += all_text.count('disease-centric')
 lag_indicators['gray_market'] += all_text.count('gray market')
 lag_indicators['gray_market'] += all_text.count('unregulated')
 lag_indicators['policy_friction'] += report['pressure_scores'].get('Policy friction pressure', 0)

 # Normalize by total primitive count
 if total_primitives > 0:
 lag_score = sum(lag_indicators.values()) / total_primitives * 10
 else:
 lag_score = 0

 return min(lag_score, 10.0) # Cap at 10

def calculate_hidden_layer_governance(
 self,
 reports: List[Dict]
) -> float:
 """
 F_hidden = k2 × (Unregulated_Market_Volume / Total_Market_Volume)

 Measures: How much economic activity occurs outside visible governance?

 Indicators:
 - Direct-to-consumer infrastructure mentions
 - "Outside traditional system" patterns
 - Private-pay / subscription models
 - "Insurance won't cover" frequency
 """

 hidden_indicators = {
 'dtc_infrastructure': 0,
 'outside_traditional': 0,
 'private_pay': 0,
 'insurance_exclusion': 0
 }

```

```

total_primitives = 0

for report in reports:
 all_text = ''.join(
 report['primitives']['fear'] +
 report['primitives']['threat'] +
 report['primitives']['response']
).lower()

 total_primitives += len(report['primitives']['fear']) + \
 len(report['primitives']['threat']) + \
 len(report['primitives']['response'])

 hidden_indicators['dttc_infrastructure'] += all_text.count('direct-to-consumer')
 hidden_indicators['dttc_infrastructure'] += all_text.count('subscription')
 hidden_indicators['outside_traditional'] += all_text.count('outside')
 hidden_indicators['outside_traditional'] += all_text.count('alternative infrastructure')
 hidden_indicators['private_pay'] += all_text.count('out-of-pocket')
 hidden_indicators['private_pay'] += all_text.count('private-pay')
 hidden_indicators['insurance_exclusion'] += all_text.count("won't cover")
 hidden_indicators['insurance_exclusion'] += all_text.count("doesn't cover")

if total_primitives > 0:
 hidden_score = sum(hidden_indicators.values()) / total_primitives * 10
else:
 hidden_score = 0

return min(hidden_score, 10.0)

def calculate_evidence_gaps(
 self,
 reports: List[Dict]
) -> float:
 """
 F_evidence = k3 × (Capital_Deployed / Clinical_Evidence_Base)

 Measures: Capital deployment outpacing scientific validation?

 Indicators:
 - "Clinical trial" absence vs. "capital deployed"
 - "Limited evidence" / "lacks backing" mentions
 - "Snake oil" / "noise" patterns
 - Investment amounts vs. efficacy proof
 """

 evidence_indicators = {
 'limited_evidence': 0,
 'lacks_backing': 0,
 'clinical_gap': 0,
 'capital_vs_evidence': 0
 }

 total_primitives = 0

 for report in reports:
 all_text = ''.join(
 report['primitives']['fear'] +

```

```

 report['primitives']['threat'] +
 report['primitives']['response']
).lower()

 total_primitives += len(report['primitives']['fear']) + \
 len(report['primitives']['threat']) + \
 len(report['primitives']['response'])

 evidence_indicators['limited_evidence'] += all_text.count('limited evidence')
 evidence_indicators['limited_evidence'] += all_text.count('lack of evidence')
 evidence_indicators['lacks_backing'] += all_text.count('lacks backing')
 evidence_indicators['lacks_backing'] += all_text.count('no clinical trial')
 evidence_indicators['clinical_gap'] += all_text.count('evidence gap')
 evidence_indicators['clinical_gap'] += all_text.count('unproven')
 evidence_indicators['capital_vs_evidence'] += all_text.count('snake oil')
 evidence_indicators['capital_vs_evidence'] += all_text.count('more noise')

 # Check for capital deployment without evidence
 if 'billion' in all_text or 'capital' in all_text:
 if 'clinical trial' not in all_text and 'evidence' not in all_text:
 evidence_indicators['capital_vs_evidence'] += 1

 if total_primitives > 0:
 evidence_score = sum(evidence_indicators.values()) / total_primitives * 10
 else:
 evidence_score = 0

 return min(evidence_score, 10.0)

def calculate_fragmentation(
 self,
 reports: List[Dict]
) -> float:
 """

$$F_{\text{fragment}} = k_4 \times \Sigma(\text{Incompatible_Systems_Count})$$

 Measures: System bifurcation and interoperability failure

 Indicators:
 - "Fragmented" / "bifurcated" mentions
 - "Incompatible" / "siloeed" patterns
 - "Two-tier system" / "parallel infrastructure"
 - Multiple proprietary platforms mentioned
 """

 fragmentation_indicators = {
 'system_bifurcation': 0,
 'incompatibility': 0,
 'parallel_systems': 0,
 'proprietary_silos': 0
 }

 total_primitives = 0

 for report in reports:
 all_text = ''.join(
 report['primitives']['fear'] +

```

```

 report['primitives']['threat'] +
 report['primitives']['response']
).lower()

 total_primitives += len(report['primitives']['fear']) + \
 len(report['primitives']['threat']) + \
 len(report['primitives']['response'])

 fragmentation_indicators['system_bifurcation'] += all_text.count('fragmented')
 fragmentation_indicators['system_bifurcation'] += all_text.count('bifurcated')
 fragmentation_indicators['incompatibility'] += all_text.count('incompatible')
 fragmentation_indicators['incompatibility'] += all_text.count('siloes')
 fragmentation_indicators['incompatibility'] += all_text.count('interoperability')
 fragmentation_indicators['parallel_systems'] += all_text.count('two-tier')
 fragmentation_indicators['parallel_systems'] += all_text.count('parallel infrastructure')
 fragmentation_indicators['parallel_systems'] += all_text.count('alternative infrastructure')
 fragmentation_indicators['proprietary_silos'] += all_text.count('proprietary')

 if total_primitives > 0:
 fragment_score = sum(fragmentation_indicators.values()) / total_primitives * 10
 else:
 fragment_score = 0

 return min(fragment_score, 10.0)

def calculate_two_tier_outcomes(
 self,
 reports: List[Dict]
) -> float:
 """

$$F_{inequality} = k_5 \times (\text{Elite_Access} - \text{Mass_Access})$$

 Measures: Access inequality and market segmentation severity

 Indicators:
 - "$50,000 membership" type mentions
 - "Wealthy" / "high-net-worth" patterns
 - "Accessible only to" phrases
 - Price tiers mentioned ($50K vs. $365 models)
 """

 inequality_indicators = {
 'premium_pricing': 0,
 'wealth_concentration': 0,
 'access_barriers': 0,
 'tier_disparity': 0
 }

 total_primitives = 0
 price_mentions = []

 for report in reports:
 all_text = ''.join(
 report['primitives']['fear'] +
 report['primitives']['threat'] +
 report['primitives']['response']
).lower()

```

```

total_primitives += len(report['primitives']['fear']) + \
 len(report['primitives']['threat']) + \
 len(report['primitives']['response'])

Extract prices
price_pattern = r'\$[\d,]+(?:\d{3})*(?:\.\d{2})?'
prices_found = re.findall(price_pattern, all_text)
price_mentions.extend([float(p.replace('$', '').replace(',', '')) for p in prices_found])

inequality_indicators['premium_pricing'] += len([p for p in prices_found if float(p.replace('$',
 '').replace(',', '')) > 10000])
inequality_indicators['wealth_concentration'] += all_text.count('wealthy')
inequality_indicators['wealth_concentration'] += all_text.count('high-net-worth')
inequality_indicators['wealth_concentration'] += all_text.count('elite')
inequality_indicators['access_barriers'] += all_text.count('accessible only to')
inequality_indicators['access_barriers'] += all_text.count('out of reach')
inequality_indicators['tier_disparity'] += all_text.count('two-tier')
inequality_indicators['tier_disparity'] += all_text.count('inequality')

Calculate price dispersion if available
if len(price_mentions) > 1:
 price_std = np.std(price_mentions)
 price_range = max(price_mentions) - min(price_mentions)
 inequality_indicators['tier_disparity'] += (price_range / 10000) # Normalize

if total_primitives > 0:
 inequality_score = sum(inequality_indicators.values()) / total_primitives * 10
else:
 inequality_score = 0

return min(inequality_score, 10.0)

def calculate_narrative_collapse(
 self,
 reports: List[Dict]
) -> float:
 """

$$F_{collapse} = k_6 \times (Fear_Score + Threat_Score - Response_Score)$$

 Measures: Imbalance between problem identification and solution deployment

 Indicators:
 - Fear score vs. Response score ratio
 - Threat score vs. Response score ratio
 - "No clear solution" patterns
 - Escalation probability
 """

 total_fear = 0
 total_threat = 0
 total_response = 0
 total_escalation_prob = 0
 report_count = len(reports)

 for report in reports:
 total_fear += report.get('narrative_scores', {}).get('Fear score', 0)

```



```

total_threat += report.get('narrative_scores', {}).get('Threat score', 0)
total_response += report.get('narrative_scores', {}).get('Response score', 0)
total_escalation_prob += report.get('regime_probabilities', {}).get('Escalation', 0)

if report_count > 0:
 avg_fear = total_fear / report_count
 avg_threat = total_threat / report_count
 avg_response = total_response / report_count
 avg_escalation = total_escalation_prob / report_count
else:
 return 0.0

Narrative collapse = (Fear + Threat) - Response + Escalation_Probability
collapse_score = ((avg_fear + avg_threat) / 2 - avg_response) + (avg_escalation * 10)

return max(min(collapse_score, 10.0), 0.0) # Bound between 0-10

def calculate_total_strain(
 self,
 reports: List[Dict]
) -> InfrastructureStrain:
 """
 Calculate comprehensive infrastructure strain across all dimensions

 Total Strain Formula:
 $\sigma_{total} = \sqrt{(F_{lag}^2 + F_{hidden}^2 + F_{evidence}^2 + F_{fragment}^2 + F_{inequality}^2 + F_{collapse}^2)}$

 Criticality Score:
 $C = (\sigma_{total} / \sigma_{max}) \times \max(\text{individual_dimension_scores})$

 Failure Probability:
 $P_{failure} = 1 - e^{(-\sigma_{total}/k_{material})}$
 """

 # Calculate each strain dimension
 lag = self.calculate_legacy_framework_lag(reports)
 hidden = self.calculate_hidden_layer_governance(reports)
 evidence = self.calculate_evidence_gaps(reports)
 fragment = self.calculate_fragmentation(reports)
 inequality = self.calculate_two_tier_outcomes(reports)
 collapse = self.calculate_narrative_collapse(reports)

 # Total strain (vector magnitude)
 total_strain = np.sqrt(
 lag**2 + hidden**2 + evidence**2 +
 fragment**2 + inequality**2 + collapse**2
)

 # Criticality score (0-10 scale)
 max_possible_strain = np.sqrt(6 * 10**2) # If all dimensions at 10
 criticality = (total_strain / max_possible_strain) * max(
 lag, hidden, evidence, fragment, inequality, collapse
)

 # Failure probability (material science exponential model)
 k_material = 15 # Material constant (tunable based on historical data)
 failure_prob = 1 - np.exp(-total_strain / k_material)

```

```

strain_object = InfrastructureStrain(
 legacy_framework_lag=lag,
 hidden_layer_governance=hidden,
 evidence_gaps=evidence,
 fragmentation=fragment,
 two_tier_outcomes=inequality,
 narrative_collapse=collapse,
 total_strain=total_strain,
 criticality_score=criticality,
 failure_probability=failure_prob
)

self.strain_history.append(strain_object)

return strain_object

```

```
class StrainVisualization:
```

```
 """Generate visualizations of infrastructure strain"""
```

```
 @staticmethod
```

```
 def generate_strain_radar(strain: InfrastructureStrain) -> str:
```

```
 """Generate ASCII radar chart of strain dimensions"""
```

```

 dimensions = [
 ('Legacy Framework Lag', strain.legacy_framework_lag),
 ('Hidden Layer Governance', strain.hidden_layer_governance),
 ('Evidence Gaps', strain.evidence_gaps),
 ('Fragmentation', strain.fragmentation),
 ('Two-Tier Outcomes', strain.two_tier_outcomes),
 ('Narrative Collapse', strain.narrative_collapse)
]

```

```

 output = ["\n## Infrastructure Strain Radar\n"]
 output.append("` ` ` `")
 output.append("Dimension Score [0-----5-----10]")
 output.append("—" * 60)

```

```

 for name, score in dimensions:
 bar_length = int((score / 10) * 30)
 bar = "█" * bar_length + "░" * (30 - bar_length)
 output.append(f"{name:28} {score:4.1f} {bar}")

```

```

 output.append("—" * 60)
 output.append(f"{'TOTAL STRAIN':28} {strain.total_strain:4.1f}")
 output.append(f"{'CRITICALITY':28} {strain.criticality_score:4.1f}")
 output.append(f"{'FAILURE PROBABILITY':28} {strain.failure_probability*100:4.1f}%")
 output.append("` ` ` ` \n")

```

```
 return '\n'.join(output)
```

```
 @staticmethod
```

```
 def generate_force_diagram(strain: InfrastructureStrain) -> str:
```

```
 """Generate ASCII force diagram showing strain vectors"""
```

```

 output = ["\n## Infrastructure Force Diagram\n"]
 output.append("` ` ` `")

```

```

output.append(" INFRASTRUCTURE SYSTEM")
output.append(" (equilibrium)")
output.append(" |")
output.append(" ")

Legacy Framework Lag force
lag_arrows = "→" * int(strain.legacy_framework_lag / 2)
output.append(f" {lag_arrows} F_lag ({strain.legacy_framework_lag:.1f})")
output.append(" |")

Hidden Layer Governance force
hidden_arrows = "←" * int(strain.hidden_layer_governance / 2)
output.append(f" | {hidden_arrows} F_hidden ({strain.hidden_layer_governance:.1f})")
output.append(" |")

Evidence Gaps force
evidence_arrows = "↑" * int(strain.evidence_gaps / 5)
for arrow in evidence_arrows:
 output.append(f" {arrow}")
output.append(f" | F_evidence ({strain.evidence_gaps:.1f})")

System node
output.append(" •")
output.append(f" / | \\\")

Fragmentation force
fragment_arrows = "↓" * int(strain.fragmentation / 5)
output.append(f" / | \\\ F_fragment ({strain.fragmentation:.1f})")
for arrow in fragment_arrows:
 output.append(f" {arrow}")

output.append(" |")

Two-Tier Outcomes force
inequality_arrows = "≥" * int(strain.two_tier_outcomes / 2)
output.append(f" {inequality_arrows} F_inequality ({strain.two_tier_outcomes:.1f})")

output.append(" |")

Narrative Collapse force
collapse_arrows = "⊗" * int(strain.narrative_collapse / 3)
output.append(f" | {collapse_arrows} F_collapse ({strain.narrative_collapse:.1f})")

output.append(" |")
output.append(" (deformed state)")
output.append("```\n")

return '\n'.join(output)

@staticmethod
def generate_strain_interpretation(strain: InfrastructureStrain) -> str:
 """Generate natural language interpretation of strain state"""

 output = ["\n## Strain Physics Interpretation\n"]

 # Dominant force identification
 forces = {

```

```

'Legacy Framework Lag': strain.legacy_framework_lag,
'Hidden Layer Governance': strain.hidden_layer_governance,
'Evidence Gaps': strain.evidence_gaps,
'Fragmentation': strain.fragmentation,
'Two-Tier Outcomes': strain.two_tier_outcomes,
'Narrative Collapse': strain.narrative_collapse
}

dominant_force = max(forces.items(), key=lambda x: x[1])

output.append(f"**Dominant Strain Vector:** {dominant_force[0]} ({dominant_force[1]:.1f}/10)\n")

Total strain interpretation
if strain.total_strain < 10:
 strain_level = "LOW"
 interpretation = "Infrastructure operating near equilibrium with manageable stress."
elif strain.total_strain < 15:
 strain_level = "MODERATE"
 interpretation = "Infrastructure experiencing elastic deformation; reversible if forces reduce."
elif strain.total_strain < 20:
 strain_level = "HIGH"
 interpretation = "Infrastructure in plastic deformation zone; permanent changes likely."
else:
 strain_level = "CRITICAL"
 interpretation = "Infrastructure approaching failure threshold; regime change imminent."

output.append(f"**Total Strain Level:** {strain_level} ({strain.total_strain:.1f}/{np.sqrt(6*10**2):.1f}
max)\n")
output.append(f"*{interpretation}*\n")

Criticality assessment
if strain.criticality_score < 3:
 criticality_level = "Stable"
elif strain.criticality_score < 6:
 criticality_level = "Elevated"
elif strain.criticality_score < 8:
 criticality_level = "Severe"
else:
 criticality_level = "Extreme"

output.append(f"**Criticality Assessment:** {criticality_level} ({strain.criticality_score:.1f}/10)\n")

Failure probability interpretation
if strain.failure_probability < 0.15:
 failure_risk = "LOW RISK"
 failure_note = "System resilient to current stress levels."
elif strain.failure_probability < 0.35:
 failure_risk = "MODERATE RISK"
 failure_note = "System vulnerable if strain increases; monitoring required."
elif strain.failure_probability < 0.60:
 failure_risk = "HIGH RISK"
 failure_note = "System failure likely without stress reduction; immediate intervention needed."
else:
 failure_risk = "EXTREME RISK"
 failure_note = "System failure imminent; emergency measures required."

output.append(f"**Failure Probability:** {failure_risk} ({strain.failure_probability*100:.1f}%)\n")

```

```

output.append(f"*{failure_note}*\n")

Dimensional breakdown
output.append("\n### Force-by-Force Analysis:\n")

for force_name, force_value in forces.items():
 if force_value > 7:
 status = " CRITICAL"
 elif force_value > 5:
 status = " ELEVATED"
 elif force_value > 3:
 status = " MODERATE"
 else:
 status = " STABLE"

 output.append(f"- **{force_name}**: ** {status} ({force_value:.1f}/10)")

return '\n'.join(output)

class CrossReportAnalysisEngine:
 """
 Master orchestration class combining primitive extraction and strain physics
 """

 def __init__(self):
 self.primitive_extractor = CrossReportPrimitiveExtractor()
 self.strain_physics = InfrastructureStrainPhysics()
 self.visualization = StrainVisualization()

 def analyze_report_collection(
 self,
 reports: List[Dict]
) -> Dict[str, any]:
 """
 Comprehensive cross-report analysis

 Returns:
 {
 'primitive_clusters': Dict of clustered primitives,
 'infrastructure_strain': InfrastructureStrain object,
 'visualizations': {
 'radar': str (ASCII radar chart),
 'force_diagram': str (ASCII force diagram),
 'interpretation': str (natural language analysis)
 },
 'recommendations': List[str]
 }
 """

 # Extract common primitives
 print(" Extracting cross-report primitives...")
 primitive_clusters = self.primitive_extractor.extract_primitives_from_reports(reports)

 # Calculate infrastructure strain
 print(" Calculating infrastructure strain physics...")
 strain = self.strain_physics.calculate_total_strain(reports)

```

```

Generate visualizations
print(" Generating strain visualizations...")
radar = self.visualization.generate_strain_radar(strain)
force_diagram = self.visualization.generate_force_diagram(strain)
interpretation = self.visualization.generate_strain_interpretation(strain)

Generate recommendations based on strain profile
recommendations = self._generate_recommendations(strain, primitive_clusters)

return {
 'primitive_clusters': primitive_clusters,
 'infrastructure_strain': strain,
 'visualizations': {
 'radar': radar,
 'force_diagram': force_diagram,
 'interpretation': interpretation
 },
 'recommendations': recommendations
}

def _generate_recommendations(
 self,
 strain: InfrastructureStrain,
 primitives: Dict[str, List[PrimitiveCluster]]
) -> List[str]:
 """Generate actionable recommendations based on strain analysis"""

 recommendations = []

 # Legacy Framework Lag recommendations
 if strain.legacy_framework_lag > 7:
 recommendations.append(
 "**URGENT - Legacy Framework Lag:** Regulatory frameworks critically obsolete. "
 "AVOID long-term commitments in heavily regulated sectors until policy clarity emerges. "
 "FOCUS on sectors with established regulatory pathways or self-regulatory models."
)
 elif strain.legacy_framework_lag > 5:
 recommendations.append(
 "**CAUTION - Legacy Framework Lag:** Regulatory uncertainty elevated. "
 "HEDGE infrastructure positions with defensive assets. MONITOR policy developments closely."
)

 # Hidden Layer Governance recommendations
 if strain.hidden_layer_governance > 7:
 recommendations.append(
 "**URGENT - Hidden Layer Governance:** Massive unregulated market activity. "
 "HIGH RISK of regulatory crack

```